Full Stack Assignment – 19

Assignment ID: FSD-19
Title: Meeting Scheduling Management System
Duration: 3 Hours

1️⃣ Objective

Design and develop a **Meeting Scheduling Management System** that allows organizers to schedule meetings and participants to view them.
The system must prevent participant conflicts and ensure **data persistence using a database**.

2️⃣ User Roles

ORGANIZER

- Register and log in to the system
- Create meetings
- Add participants to meetings
- View all scheduled meetings

PARTICIPANT

- Register and log in to the system
- View meetings they are invited to

3️⃣ Functional Requirements

Authentication & Authorization

- Implement user registration and login
- Use **JWT-based authentication**
- Protect all backend APIs using JWT
- Enforce **role-based access control** on APIs and frontend views

Meeting Management

- ORGANIZER can create meetings with date and time range
- Meeting data must be stored in the database
- Participant assignments must be persisted

Participant Scheduling

- ORGANIZER assigns participants to meetings
- PARTICIPANT can view assigned meetings
- Scheduling data must reflect real-time database state

4️⃣ Business Rule (Critical)

⚠️ **A participant must not be scheduled for overlapping meetings.**

Any scheduling attempt that creates a time conflict for a participant must be rejected.

5️⃣ Technical Constraints (Mandatory)

- All meeting and participation data **must persist in a database**
- Hardcoded or in-memory storage is **not allowed**
- Backend APIs must reflect actual database state
- JWT authentication and role validation are mandatory
- Use environment variables for sensitive configuration

6️⃣ Frontend Requirements

- Login and registration pages
- Role-based dashboards for ORGANIZER and PARTICIPANT
- Meeting creation and participant assignment view for ORGANIZER
- Meeting listing view for PARTICIPANT

UI design is not evaluated; logic and correctness matter.

7️⃣ Submission Requirements (Strict)

GitHub Repository

- Backend source code
- Frontend source code
- Proper folder structure
- Clear and meaningful commit history

Live Deployment

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

README.md (Mandatory)

The README file must include:

- Project overview
- Tech stack used
- User roles and permissions
- API endpoints

- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

8️⃣ Evaluation Note

Evaluation focuses on **conflict detection, role enforcement, and end-to-end integration**.